

리눅스 커널 입문 가이드

기반 지식 쌓기부터 커뮤니티 참여까지

이 문서는 커널 개발자에게는 익숙하지만 입문자에게는 당연하지 않은 것들을 정리한 것이다. 크게 세 부분으로 나뉜다. 첫째, 입문자가 마주치는 단계별 어려움과 그 대처법. 둘째, 잘 모르는 서브시스템을 분석하는 방법. 셋째, 실제로 커널 개발에 참여하는 방법.

Hyeonggon Yoo, "초보자가 말하는, 초보자를 위한 커널 입문 가이드" 발표 내용을 입문 학습용으로 재구성

1. 입문자가 마주하는 어려움들

커널 개발에 익숙해지는 과정은 대체로 다음 순서를 밟는다.

컴공 기반 지식 쌓기 → 커널 환경에 익숙해지기 → 첫 번째 기여 → 메일링 리스트 구독 및 관심 서브시스템 분석 → 활발한 참여와 내 분야에서의 전문성

1.1 1단계 — 컴퓨터 공학 기반 지식

너무 당연한 말이지만, 입문할 때는 자기가 뭘 모르는지조차 모른다는 게 문제다. 운영체제는 여러 분야의 배경 지식을 동시에 요구한다. 커널부터 바로 시작하고 싶어도 결국은 아래를 다 거쳐야 한다.

- C언어
- git
- 자료구조와 알고리즘
- 컴퓨터구조
- 시스템 프로그래밍
- 운영체제

전부를 엄청나게 잘 할 필요는 없다. 다만 어느 정도는 다 알아야 한다. 한 번에 너무 많은 단계를 건너뛰면 아무리 읽어도 이해가 되지 않는다. 조금해하지 말 것.

1.2 2단계 — 커널 환경에 익숙해지기

“리눅스라는 운영체제는 어떤 방식으로 구현되어 있는가”를 스스로 설명할 수 있어야 한다. 아래 네 축을 기준으로 점검해 보면 좋다.

실행 컨텍스트

CPU는 임의의 시점에 어떤 작업을 수행하고 있는가? (프로세스, 인터럽트 핸들러, 커널 스레드, 후반부 작업 등)

선점 모델

- 인터럽트 핸들러는 선점될 수 있는가?
- 커널 스레드는 다른 스레드에 의해 선점될 수 있는가?
- 인터럽트 핸들러가 프로세스 컨텍스트에서 실행되는가?

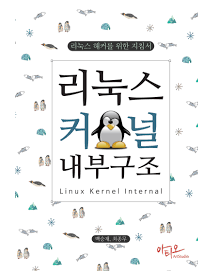
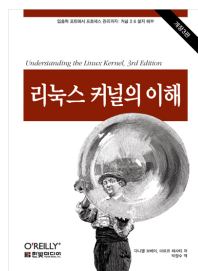
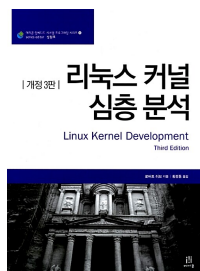
동기화 메커니즘

- 여러 CPU 간, 혹은 스레드와 인터럽트 핸들러 간의 공유 자원을 어떻게 관리하는가?

주소 공간

- 리눅스는 32/64비트 주소 공간을 어떻게 쓰는가?
- 프로세스와 커널은 주소 공간을 어떻게 나눠 쓰는가?
- 커널은 임의의 물리 메모리에 접근할 때 어떤 가상 주소를 쓰는가?

이 단계에서는 특정 서브시스템을 깊게 파기보다, 커널 전반을 다루는 책이나 문서로 넓고 얇은 지도를 먼저 만드는 편이 효율적이다.



커널 전반을 다루는 입문서들 — 한 권을 정해 통독하는 것으로 이 단계를 시작하면 된다.

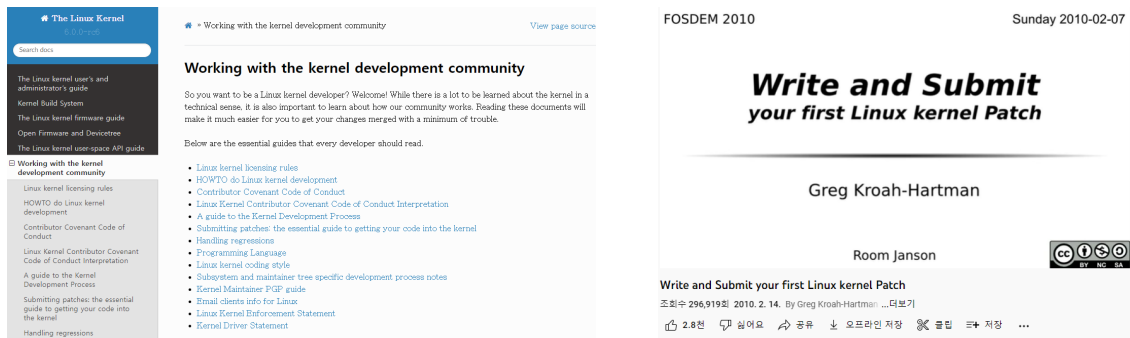
1.3 3단계 — 첫 번째 기여

책을 다 읽고 나면 기여를 해보고 싶어진다. 패치를 보내기 전에 먼저 할 일이 있다.

1. **Documentation/process**를 먼저 훑어본다. 개발 방식, 개발 주기(릴리즈 사이클), stable 커널 관리 방식 등 읽을거리가 많고 그 자체로 재미있다.

2. 간단한 패치로 감을 잡는다. 처음에는 잘 아는 분야가 없으므로 coding style fix 같은 단순한 패치로 제출 절차 자체에 익숙해지는 것이 목적이다.
3. 다만 trivial한 style fix는 한두 번으로 끝낸다. 절차를 익히는 수단일 뿐, 계속 반복하면 커뮤니티에서 좋게 보지 않는다.

Greg Kroah-Hartman이 drivers/staging에 첫 패치를 보내는 과정을 보여주는 튜토리얼 영상이 공개되어 있다. 첫 제출 절차를 눈으로 따라가기에 좋다.



왼쪽: 커널 문서의 개발 프로세스 안내. 오른쪽: Greg Kroah-Hartman의 첫 패치 제출 튜토리얼 (FOSDEM 2010).

1.4 4단계 — “이제 뭘 하지?”

커널에도 익숙해졌고 기여 방법도 대충 알겠는데, 정작 무엇을 해야 할지 몰라 방황하는 시기가 온다. 사실 이건 커널과 무관하게 누구나 전문 분야를 정하기 전에 겪는 고민이다.

“나는 뭘 해야 하지?” (×)

“나는 뭘 하고 싶지?” (○)

어차피 정해진 답은 없다. 처음에 무엇을 고르는지는 생각보다 중요하지 않고, 일단 아무거나 해본 뒤에 나중에 바뀌어도 된다.

한 가지 분야를 1년쯤 파다 보면 판단 기준이 생긴다. 분야를 고를 때 스스로에게 물어볼 것.

- 활발하게 개발되는 분야인가?
- 경쟁이 너무 세지는 않은가?
- 흥미로운 문제가 많은가?
- 재미있을까?

2. 잘 모르는 서브시스템 분석하기

처음에는 뭐든지 다 어렵다. 아는 건 별로 없지만 차근차근 하면 된다.

2.1 코드보다 설계가 먼저다

무작정 코드부터 읽으면 한계에 부딪힌다. 아무리 읽어도 이해가 되지 않는 이유는 대개 한 번에 너무 많은 것을 건너뛰었기 때문이다. 가장 쉬운 것부터 차례로 올라가야 한다.



아무리 읽어도 이해가 되지 않는 시기는 누구에게나 온다.

2.2 큰 그림 그리기

분석 대상이 무엇을 목적으로 하는 물건인지 한 문장으로 말할 수 있어야 한다.

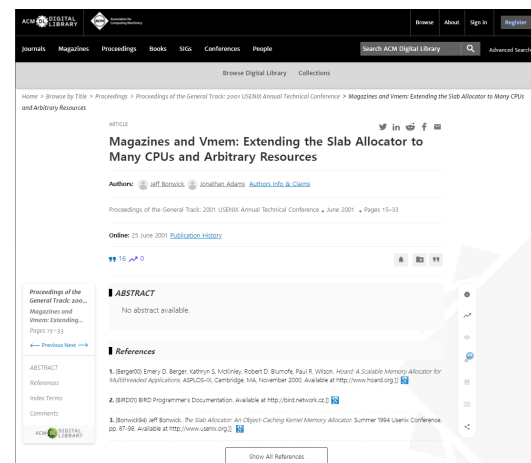
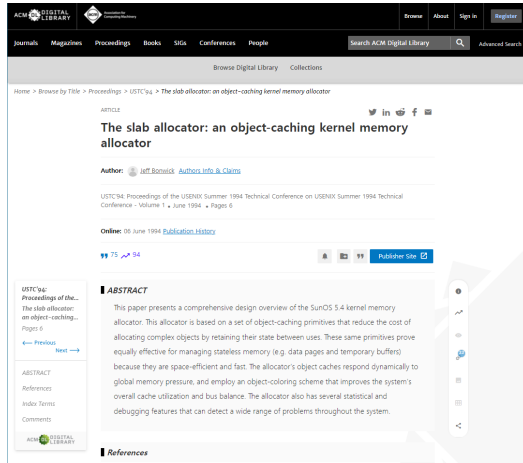
- 슬랩 할당자 — 크기가 작은 객체를 내부 단편화 적게, 빠르고 캐시 친화적으로 할당
- 스케줄러 — 적은 오버헤드로 다음 실행할 태스크를 고르되 throughput, latency, power 등을 만족
- 페이지 캐시 — 디스크 IO를 최소화하고 최대한 메모리에서 읽도록 캐싱
- io_uring — 링 버퍼를 써서 system call 없이 적은 오버헤드로 IO 수행
- PFRA (Page Frame Replacement Algorithm) — 메모리가 부족할 때 제일 안 쓸 법한 메모리를 해제해 여유 확보

2.3 자료 조사

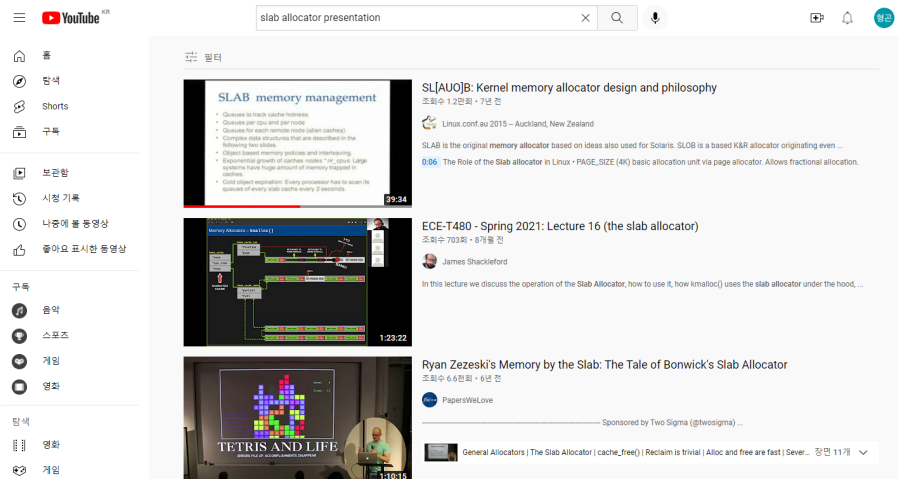
코드 분석을 시작하기 전에, 전체 흐름과 설계를 많이 알수록 좋다. 다음 순서로 찾아본다.

1. 커널 문서 — Documentation/ 아래의 해당 서브시스템 문서
2. 논문 — 설계의 출발점이 된 원 논문. (슬랩 할당자의 경우 주요 논문이 두 편 있다)
3. 컨퍼런스 발표 영상 — 메인테이너가 직접 구현을 설명하는 경우가 많다
4. 블로그, 분석 글 — 유명한 서브시스템일수록 잘 정리된 글이 많다

목표는 코드를 읽기 전에 서브시스템의 구조와 big picture를 최대한 확보하는 것이다.



슬랩 할당자의 설계를 다룬 두 편의 원 논문.



메인테이너가 직접 구현을 설명하는 컨퍼런스 발표 영상도 여러 편 찾을 수 있다.

2.4 코드 분석 — 최신 메인라인은 너무 복잡하다

최신 구현을 바로 분석하려 하면 벅차다. 예를 들어 리눅스 6.0의 SLUB 할당자 하나가 아래를 전부 감당한다.

- 내부 단편화 최소화 / 캐시 친화적 배치
- NUMA 아키텍처 지원
- lock contention 최소화 (cpu partial)
- 실시간 커널에서의 반응성
- 메모리 어카운팅(memcg) 지원
- 새너타이저 지원 (KASAN, KMSAN 등)
- Fault injection 지원 (failslab)

- 디버깅 모드, 저비용 통계, sysfs 인터페이스

가장 단순한 버전부터 보고, 히스토리를 천천히 따라가라. SLUB은 2007년에 처음 도입되었으므로 그 즈음의 코드는 훨씬 단순하다. 초기 버전에는 위 목록 중 절반도 들어 있지 않다.



index : kernel/git/torvalds/linux.git

Linux kernel source tree

[about](#) [summary](#) [refs](#) [log](#) [tree](#) **[commit](#)** [diff](#) [stats](#)

author	Christoph Lameter <clameter@sgi.com>	2007-05-06 14:49:36 -0700
committer	Linus Torvalds <torvalds@woody.linux-foundation.org>	2007-05-07 12:12:53 -0700
commit	81819f0fc8285a2a5a921c019e3e3d7b6169d225 (patch)	
tree	47e3da44d3ef6c74ceae6c3771b191b46467bb48	
parent	543691a6cd70b606dd9bed5e77b120c5d9c5c506 (diff)	
download	linux-81819f0fc828.tar.gz	

SLUB core

This is a new slab allocator which was motivated by the complexity of the existing code in mm/slab.c. It attempts to address a variety of concerns with the existing implementation.

A. Management of object queues

A particular concern was the complex management of the numerous object queues in SLAB. SLUB has no such queues. Instead we dedicate a slab for each allocating CPU and use objects from a slab directly instead of queueing them up.

B. Storage overhead of object queues

SLAB Object queues exist per node, per CPU. The alien cache queue even has a queue array that contain a queue for each processor on each node. For very large systems the number of queues and the number of objects that may be caught in those queues grows exponentially. On our systems with 1k nodes / processors we have several gigabytes just tied up for storing references to objects for those queues. This does not include the objects that could be on those queues. One fears that the whole memory of the machine could one day be consumed by those queues.

C. SLAB meta data overhead

SLAB has overhead at the beginning of each slab. This means that data cannot be naturally aligned at the beginning of a slab block. SLUB keeps all meta data in the corresponding page_struct. Objects can be naturally aligned in the slab. F.e. a 128 byte object will be aligned at 128 byte boundaries and can fit tightly into a 4k page with no bytes left over. SLAB cannot do this.

D. SLAB has a complex cache reaper

SLUB does not need a cache reaper for UP systems. On SMP systems the per CPU slab may be pushed back into partial list but that operation is simple and does not require an iteration over a list of objects. SLAB expires per CPU, shared and alien object queues during cache reaping which may cause strange hold offs.

SLUB을 처음 도입한 2007년 커밋. 구현 의도가 커밋 메시지에 그대로 설명되어 있다.

이후에는 git 히스토리와 메일링 리스트 아카이브(lore.kernel.org)를 활용해 기능이 왜, 어떻게 추가됐는지를 따라간다. 코드를 읽다 생긴 의문 — 예를 들어 “왜 RT 커널에서는 fastpath를 못 쓰지? 락이 어떻게 바뀐 거지?” — 은 대부분 그 변경을 도입한 패치와 그 패치를 둘러싼 메일 스레드에 답이 있다.

```
hyeyoo@hyeyoo: ~/hygoni

3 author Vlastimil Babka <vbabka@suse.cz> Sat May 22 01:59:38 2021 +0200
4 committer Vlastimil Babka <vbabka@suse.cz> Sat Sep 4 10:22:01 2021 +0200
5
6 mm, slub: convert kmem_cpu_slab protection to local_lock
7
8 Embed local_lock into struct kmem_cpu_slab and use the irq-safe versions of
9 local_lock instead of plain local_irq_save/restore. On !PREEMPT_RT that's
10 equivalent, with better lockdep visibility. On PREEMPT_RT that means better
11 preemption.
12
13 However, the cost on PREEMPT_RT is the loss of lockless fast paths which only
14 work with cpu freelist. Those are designed to detect and recover from being
15 preempted by other conflicting operations (both fast or slow path), but the
16 slow path operations assume they cannot be preempted by a fast path operation,
17 which is guaranteed naturally with disabled irqs. With local locks on
18 PREEMPT_RT, the fast paths now also need to take the local lock to avoid races.
19
20 In the allocation fastpath slab_alloc_node() we can just defer to the slowpath
21 __slab_alloc() which also works with cpu freelist, but under the local lock.
22 In the free fastpath do_slab_free() we have to add a new local lock protected
23 version of freeing to the cpu freelist, as the existing slowpath only works
24 with the page freelist.
25
26 Also update the comment about locking scheme in SLUB to reflect changes done
27 by this series.
28
29 [ Mike Galbraith <efault@gmx.de>: use local_lock() without irq in PREEMPT_RT
30   scope; debugging of RT crashes resulting in put_cpu_partial() locking changes ]
31 Signed-off-by: Vlastimil Babka <vbabka@suse.cz>
```

변경을 설명하는 패치 메일. “왜 이렇게 바뀌었는가”에 대한 1차 자료다.

3. 커널 개발에 참여하기

공부만으로 끝내기엔 아쉽다. 그런데 코드 분석과 개발은 상당히 다른 일이라 어디서부터 시작할지 막막하다.

3.1 커널 개발은 코드 작성이 전부가 아니다

중요한 것은 포지셔닝이다. 커뮤니티에는 다양한 역할이 있고 모두 각자의 몫이 있다.

- 관리자 / 메인테이너
- 리뷰어
- Regression Tracker
- Stable team, Security team
- 그냥 활발한 개발자
- 버그 리포트를 하는 사람, 그리고 버그 리포트를 하는 봇
- 기능 테스트와 성능 측정을 돌리는 봇

포지션을 잡으면 서브시스템의 한 부품이 된 듯한 감각이 생긴다.

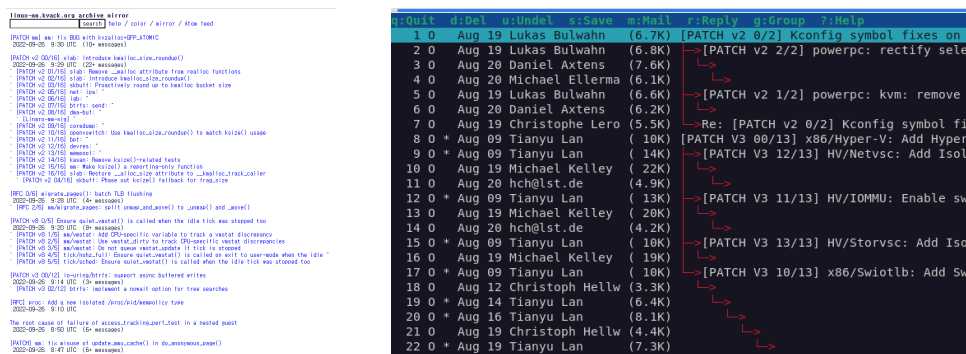
3.2 리뷰어로 시작하는 것도 나쁘지 않다

1. **남의 코드를 리뷰하며 모르는 걸 배운다.** 처음 분석할 때 전부 이해되지는 않으므로 여러 번 봐야 하는데, 리뷰를 하다 보면 잘 몰랐던 부분도 한 번씩 보게 된다. 그리고 자기 코드에 대한 질문을 싫어하는 사람은 잘 없다.
2. **코드 리뷰는 항상 부족하다.** linux-mm만 봐도 새로운 코드는 많은데 리뷰는 턱없이 모자란다. 검토되지 않고 테스트되지 않은 코드는 문제를 일으키기 쉽다. 즉 수요가 있다.
3. **처음부터 코드 기여로 시작하기는 어렵다.** 모르는 게 많을 때는 사람들이 무엇을 하는지 먼저 보고, 익숙해진 뒤에 개발하는 편이 수월하다.

3.3 메일링 리스트는 항상 모니터링

어떤 서브시스템에서 어떤 포지션을 맡든, 개발을 하려면 지금 커뮤니티에서 무슨 일이 일어나는지 알아야 한다. 커널 개발은 이메일로 진행되기 때문에 자주 읽어야 한다. 가능하면 매일 읽는 습관을 들이는 게 좋다.

- lore.kernel.org — 메일링 리스트 아카이브. 검색과 스레드 추적에 필수
- [mutt](http://mutt.org) — 터미널 기반 이메일 클라이언트. 패치 메일을 다루기에 편하다



왼쪽: lore.kernel.org 아카이브. 오른쪽: mutt로 패치 메일을 읽는 모습.

3.4 이메일 바깥에도 좋은 자료가 많다

모든 이메일을 다 읽을 수는 없다. 내 분야가 아닌 쪽의 동향은 기사나 발표로 따라가면 된다.

- 컨퍼런스 — 매년 열리는 Linux Plumbers Conference, LSFMM 등 분야별로 다양하다
- 뉴스 — 개발 소식이 자주 올라오는 LWN.net, Phoronix



한 장 요약

지금 내가 어느 단계인지 점검하는 체크리스트

기반 지식

- ☐ C, git, 자료구조/알고리즘, 컴퓨터구조, 시스템 프로그래밍, 운영체제를 어느 정도씩 알고 있다
- ☐ 모르는 부분이 나왔을 때 건너뛰지 않고 되돌아가 채운다

커널 환경

- ☐ 실행 컨텍스트(프로세스 / 인터럽트 / 커널 스레드 / 후반부 작업)를 구분할 수 있다
- ☐ 선점 모델과 동기화 메커니즘을 설명할 수 있다
- ☐ 커널과 프로세스의 주소 공간 분할을 설명할 수 있다

첫 기여

- ☐ Documentation/process를 훑어봤다
- ☐ 패치 제출 절차를 한 번 끝까지 경험해봤다

방향 정하기

- ☐ “뭘 해야 하지”가 아니라 “뭘 하고 싶지”로 질문을 바꿨다
- ☐ 후보 분야를 네 기준(활발함 / 경쟁 / 문제의 흥미도 / 재미)으로 따져봤다

서브시스템 분석

- ☐ 대상의 목적을 한 문장으로 말할 수 있다
- ☐ 코드를 읽기 전에 문서, 논문, 발표, 블로그를 먼저 훑었다
- ☐ 최신 코드가 아니라 최초 도입 시점의 단순한 버전부터 봤다
- ☐ 궁금한 변경은 git 히스토리와 lore.kernel.org에서 근거를 찾는다

커뮤니티 참여

- ☐ 관심 서브시스템의 메일링 리스트를 구독하고 주기적으로 읽는다
- ☐ 내가 맡을 수 있는 포지션을 하나 정했다 (리뷰어부터 시작해도 좋다)
- ☐ LWN.net, Phoronix, 컨퍼런스 발표로 다른 분야 동향도 챙긴다

**Don't rush things, and...
enjoy your journey!**

